

Filtro de Sobel en Python: Comparación secuencial, NumPy y Numba CPU

Institución: UNTDF

Carrera: Lic. en Sistemas, 5to año

Materia: Sistemas Paralelos

Docente: MsC. Federico González Brizzio

Abstract

Se implementó el filtro de Sobel para detección de bordes sobre imágenes RGB convertidas a escala de grises, comparando tres enfoques pedidos para la primera entrega: una versión secuencial en Python puro, una versión vectorizada con NumPy y una versión compilada/paralela en CPU con Numba. El benchmark se ejecutó sobre imágenes de 750x750, 1500x1500, 3000x3000 y 6000x6000 píxeles, con 5 corridas por combinación. Para cada método se midió por separado el tiempo de conversión RGB->gris, el tiempo de aplicación de Sobel y el tiempo total, excluyendo carga y guardado de archivos. La salida se comparó mediante el porcentaje de píxeles blancos en la imagen Sobel. En todos los tamaños, los tres métodos produjeron el mismo porcentaje de blancos, lo que indica consistencia de salida para esta métrica. NumPy y Numba CPU redujeron fuertemente los tiempos respecto del secuencial; Numba CPU obtuvo los mejores tiempos totales en 1500x1500, 3000x3000 y 6000x6000, mientras que NumPy fue levemente mejor en 750x750.

1. Introduccion

El operador de Sobel estima cambios locales de intensidad mediante dos mascarar 3x3: una para el gradiente horizontal (G_x) y otra para el gradiente vertical (G_y). Para cada píxel interior se calcula la magnitud aproximada del gradiente como:

$$|\text{grad}| = \sqrt{g_x^2 + g_y^2}$$

El resultado se recorta al rango $[0, 255]$ para obtener una imagen de bordes en escala de grises. Como el cálculo se aplica pixel por pixel, el problema es adecuado para comparar el costo de una implementación secuencial frente a implementaciones optimizadas o paralelas.

Esta entrega se limita a:

1. Versión secuencial en Python puro.
2. Version vectorizada con NumPy.
3. Version Numba CPU parallel.

Las versiones Numba GPU, PyTorch CPU y PyTorch GPU corresponden a entregas posteriores.

2. Metodologia

2.1 Equipo

Propiedad	Valor
CPU	Intel(R) Core(TM) i5-10300H CPU @ 2.50GHz
Nucleos fisicos	4
Nucleos logicos	8
RAM	15 GiB
Sistema operativo	Manjaro Linux, kernel 6.12.85-1-MANJARO
Python	3.14.4
NumPy	2.4.4
Numba	0.65.1
Pillow	12.2.0
GPU	No usada en esta entrega

Nota: se solicitó Python 3.14.3, pero en el equipo estaba disponible Python 3.14.4 como versión estable instalada. El entorno virtual usado fue `tp3/.venv`.

2.2 Algoritmos evaluados

1. **Secuencial:** conversión RGB->gris y Sobel con bucles de Python puro. No usa NumPy para el cómputo; usa bytes/bytearray como almacenamiento para evitar el costo de listas gigantes en 6000x6000.
2. **NumPy:** conversion y Sobel con operaciones vectorizadas sobre arreglos ndarray.
3. **Numba CPU:** kernels compilados con `@njit(parallel=True)` y `prange`, usando hasta 8 hilos logicos.

2.3 Parámetros experimentales

Parametro	Valor
Tamanios	750x750, 1500x1500, 3000x3000, 6000x6000
Corridas por caso	5
Metodos	secuencial, numpy, numba_cpu
Workers Numba CPU	8

Parametro	Valor
Entrada	Imágenes provistas por la catedra (IMG_0358_*)
Script de corrida	python correr_entrega1.py
CSV	resultados_sobel_entrega1.csv
Tablas Markdown	resultados_sobel_entrega1.md

La carga y el guardado de archivos no forman parte de los tiempos medidos. El cronómetro solo cubre:

1. Conversion RGB->gris.
2. Aplicación del filtro Sobel.

2.4 Metricas

- **Tiempo RGB->gris (s):** tiempo promedio de conversión de imagen RGB a escala de grises.
- **Tiempo Sobel (s):** tiempo promedio del filtro Sobel sobre la imagen gris.
- **Tiempo total (s):** suma medida de conversión y Sobel.
- **% blancos:** (pixeles con valor 255 / píxeles totales) * 100.
- **Speed-up:** tiempo_total_secuencial / tiempo_total_metodo, para el mismo tamaño.
- **Performance (%):** speed-up / workers * 100.

La columna performance (%) debe leerse como una métrica relativa definida por la consigna. Puede superar el 100% porque compara contra un baseline secuencial de Python, no contra un límite físico de uso de CPU.

3. Resultados

Los valores siguientes provienen de resultados_sobel_entrega1.csv, con 5 corridas por combinación.

3.1 Imagen 750x750

Metodo	Tiempo RGB->gris (s)	Tiempo Sobel (s)	Tiempo total (s)	% blancos	Speed-up	Performance (%)
secuencial	0.13759 8206	0.38108 0508	0.51867 8714	0.28177 7778	1.00000 0	100.00
numpy	0.00358 4823	0.00527 3254	0.00885 8076	0.28177 7778	58.5543 28	5855.43

Metodo	Tiempo RGB->gris (s)	Tiempo Sobel (s)	Tiempo total (s)	% blancos	Speed-up	Performance (%)
numb a_cpu	0.00808 6321	0.00136 1815	0.00944 8136	0.28177 7778	54.8974 66	686.22

3.2 Imagen 1500x1500

Metodo	Tiempo RGB->gris (s)	Tiempo Sobel (s)	Tiempo total (s)	% blancos	Speed-up	Performance (%)
secue ncial	0.58917 1745	1.60783 1727	2.19700 3472	0.05995 5556	1.00000 0	100.00
numpy	0.01378 5413	0.02585 5008	0.03964 0422	0.05995 5556	55.4233 12	5542.33
numb a_cpu	0.00191 1381	0.00541 0280	0.00732 1660	0.05995 5556	300.069 022	3750.86

3.3 Imagen 3000x3000

Metodo	Tiempo RGB->gris (s)	Tiempo Sobel (s)	Tiempo total (s)	% blancos	Speed-up	Performance (%)
secue ncial	2.42550 3625	6.51263 7727	8.93814 1351	0.00136 6667	1.00000 0	100.00
numpy	0.06000 8272	0.12006 8416	0.18007 6688	0.00136 6667	49.6351 94	4963.52
numb a_cpu	0.00632 5961	0.01117 4299	0.01750 0260	0.00136 6667	510.743 340	6384.29

3.4 Imagen 6000x6000

Metodo	Tiempo RGB->gris (s)	Tiempo Sobel (s)	Tiempo total (s)	% blancos	Speed-up	Performance (%)
secue ncial	9.81372 9013	26.4605 40787	36.2742 69800	0.00000 0000	1.00000 0	100.00
numpy	1.74276 3535	2.40278 0660	4.14554 4195	0.00000 0000	8.75018 3	875.02
numb a_cpu	0.02710 0780	0.02970 5952	0.05680 6732	0.00000 0000	638.555 833	7981.95

4. Discusión

4.1 Diferencias de tiempo entre secuencial, NumPy y Numba CPU

El método secuencial fue el más lento en todos los tamaños, como era esperable por ejecutar bucles interpretados en Python. El costo crece de forma clara al aumentar la resolución: pasa de 0.519 s en 750x750 a 36.274 s en 6000x6000.

NumPy reduce fuertemente los tiempos frente al secuencial por vectorización. En 750x750 obtuvo el mejor tiempo total (0.00886 s), levemente por debajo de Numba CPU (0.00945 s). En los tamaños mayores, Numba CPU domina: 0.00732 s en 1500x1500, 0.01750 s en 3000x3000 y 0.05681 s en 6000x6000.

En el caso de 6000x6000, NumPy también mejora mucho al secuencial (4.146 s contra 36.274 s), pero Numba CPU queda muy por debajo (0.0568 s). Esto muestra que, para este núcleo puntual de Sobel, los bucles compilados y paralelos tienen una ventaja importante frente a la expresión vectorizada que crea arreglos intermedios.

4.2 Evolucion del speed-up

El speed-up de NumPy no crece monotónicamente. Fue 58.55x en 750x750, 55.42x en 1500x1500, 49.64x en 3000x3000 y baja a 8.75x en 6000x6000. La caída en el caso más grande se atribuye al costo de memoria y a los arreglos temporales que genera la versión vectorizada para gx, gy y la magnitud.

Numba CPU muestra un comportamiento más favorable al aumentar la resolución: 54.90x en 750x750, 300.07x en 1500x1500, 510.74x en 3000x3000 y 638.56x en 6000x6000. En imágenes chicas, el costo fijo de preparar/ejecutar kernels paralelos pesa más; en imágenes grandes, ese costo se amortiza y predomina el cómputo paralelizado.

4.3 Equivalencia visual y porcentaje de blancos

Los tres métodos dieron exactamente el mismo porcentaje de pixeles blancos para cada tamaño:

Tamano	% blancos
750x750	0.281777778
1500x1500	0.059955556
3000x3000	0.001366667
6000x6000	0.000000000

Esto indica que las implementaciones son consistentes para la métrica pedida. En una primera versión del código, Numba podría producir diferencias por desbordamiento al operar directamente con uint8; se corrigió casteando los pixeles a enteros con signo antes de calcular gx y gy.

El porcentaje de blancos cae con la resolución porque la métrica solo cuenta píxeles exactamente iguales a 255. Aunque la imagen Sobel contenga bordes visibles en otros niveles de gris, estos píxeles no suman como “blancos” para esta consigna.

5. Conclusion

La implementación secuencial sirve como baseline y confirma el costo alto de aplicar Sobel pixel por pixel en Python puro. NumPy mejora de forma marcada al vectorizar las operaciones y fue el método más rápido en 750x750, donde el problema es chico y los costos fijos de Numba pesan más. Para 1500x1500 en adelante, Numba CPU fue el método más rápido, con speed-ups crecientes hasta 638.56x en 6000x6000.

Las tres implementaciones fueron equivalentes en la métrica de salida exigida: el porcentaje de píxeles blancos coincide en todos los tamaños. Por lo tanto, las diferencias observadas son de rendimiento y no de resultado numérico para esta comparación.